

Lab 1: Triggering Nested Loop Join

The screenshot shows a database query interface with a query editor and a query plan output.

Query:

```
1  -- Step 1: Execute this and observe the "Nested Loop" node
2  EXPLAIN (ANALYZE, BUFFERS)
3  SELECT c.first_name, c.last_name, r.rental_date
4  FROM customer c
5  JOIN rental r ON c.customer_id = r.customer_id
6  WHERE c.customer_id = 1;
```

Query Plan:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	Nested Loop	0.28..359.16	32	21	1.220..4.850	34.00	1
2	Buffers: shared read=153 dirtied=2						
3	-> Index Scan using customer_pkey on customer c	0.28..8.29	1	17	0.315..0.316	1.00	1
4	Index Cond: (customer_id = 1)						
5	Index Searches: 1						
6	Buffers: shared read=3						
7	-> Seq Scan on rental r	0.00..350.55	32	10	0.901..4.518	34.00	1
8	Filter: (customer_id = 1)						
9	Rows Removed by Filter: 16012						
10	Buffers: shared read=150 dirtied=2						
11	Planning:						
12	Buffers: shared hit=120 read=8 dirtied=1						
13	Planning Time: 5.211 ms						
14	Execution Time: 4.881 ms						

1. Identify the Roles: Look at the plan tree. Which table is the Outer (top) and which is the Inner (bottom)?
Roles: Outer (top): Bảng customer (index scan), Inner (bottom): Bảng rental (seq scan)
2. Why Nested Loop? How many rows did the customer table return? Why does this make Hash Join "too expensive"?
Bảng customer trả về đúng 1 dòng, Hash Join đắt đỏ vì tạo Hash Table tốn chi phí CPU và bộ nhớ không cần thiết khi Outer table chỉ có 1 dòng, với 1 dòng quét trực tiếp bằng Nested Loop nhanh và nhẹ hơn.
3. Efficiency: Look at loops in the Inner Table node. Why is it 1?
Outer table customer chỉ có duy nhất 1 dòng thỏa mãn điều kiện `customer_id = 1`, nên node tìm kiếm ở inner table rental chỉ cần thực thi đúng 1 lần.

Lab 2: Triggering Hash Join

```
7
8  -- Run this and look for "Hash Join" and "Hash" nodes
9  EXPLAIN (ANALYZE, BUFFERS)
10 SELECT f.title, l.name AS language
11 FROM film f
12 JOIN language l ON f.language_id = l.language_id;
13
14 SET work_mem = '64kB';
```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

	QUERY PLAN
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=1.579..3.648 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared read=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.725..2.612 rows=1000.00 loo...
5	Buffers: shared read=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.839..0.840 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.830..0.832 rows=6.00 loo...
10	Buffers: shared read=1
11	Planning:
12	Buffers: shared hit=297 read=9 dirtied=1
13	Planning Time: 19.020 ms
14	Execution Time: 3.988 ms

1. Identify Phases: Which table is being Hashed (Built)? Which table is being Probed?
Hashed (Bult) - bảng language, Probed - bảng film
2. Memory Check: Look at the Hash node. What is the Memory Usage?
Memory usage: 9KB

```
13
14 SET work_mem = '64kB';
```

Data Output Messages Notifications

SET

Query returned successfully in 71 msec.

```

8  -- Run this and look for "Hash Join" and "Hash" nodes
9  EXPLAIN (ANALYZE, BUFFERS)
10 SELECT f.title, l.name AS language
11 FROM film f
12 JOIN language l ON f.language_id = l.language_id;
13
14 SET work_mem = '64kB';

```

Data Output Messages Notifications

Showing rows: 1 to 12 of 1

	QUERY PLAN
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.034..0.302 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.014..0.110 rows=1000.00 loops=1)
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.014..0.015 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.005..0.007 rows=6.00 loops=1)
10	Buffers: shared hit=1
11	Planning Time: 0.153 ms
12	Execution Time: 0.357 ms

3. Hypothesis: What happens if you run SET work_mem = '64kB'; and run it again? (Try it!)

Kết quả không thay đổi, vì dung lượng Hash Table chỉ có 9KB vẫn nhỏ hơn giới hạn mới là 64KB. Nếu kích thước dữ liệu vượt quá work_mem, Postgres mới tăng Batches và ghi dữ liệu ra đĩa cứng

Lab 03: Triggering Merge Join

```

14 SET work_mem = '64kB';
15
16 -- Lab 3:
17 SET enable_hashjoin = OFF;
18 SET enable_nestloop = OFF;
19
20 EXPLAIN (ANALYZE, BUFFERS)
21 SELECT f.film_id, f.title, i.inventory_id
22 FROM film f
23 JOIN inventory i ON f.film_id = i.film_id
24 ORDER BY f.film_id;
25
26 RESET ALL;

```

Data Output Messages Notifications

Showing rows: 1 to 19 of 1

	QUERY PLAN
1	Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=3.116..4.660 rows=4581.00 loops=1)
2	Merge Cond: (f.film_id = i.film_id)
3	Buffers: shared hit=87 read=26 dirtied=2, temp read=10 written=12
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.006..0.203 rows=1000.00 loops=1)
5	Index Searches: 1
6	Buffers: shared hit=84 read=1
7	-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=3.105..3.803 rows=4581.00 loops=1)
8	Storage: Memory Maximum Storage: 17kB
9	Buffers: shared hit=3 read=25 dirtied=2, temp read=10 written=12
10	-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=3.100..3.404 rows=4581.00 loops=1)
11	Sort Key: i.film_id
12	Sort Method: external merge Disk: 80kB
13	Buffers: shared hit=3 read=25 dirtied=2, temp read=10 written=12
14	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=1.062..1.819 rows=4581.00 loops=1)
15	Buffers: shared read=25 dirtied=2
16	Planning:
17	Buffers: shared hit=89 read=5
18	Planning Time: 14.108 ms
19	Execution Time: 11.929 ms

1. Structure: Do you see a Sort node? Or did it use Index Scan to get sorted data for free?

Có thấy Sort node cho bảng inventory, bảng film dùng Index Scan nên lấy dữ liệu đã được sắp xếp miễn phí, nhưng bảng inventory quét qua Seq Scan nên Postgres vẫn bắt buộc phải thêm node Sort để sắp xếp bảng inventory theo film_id trước khi Merge Join

```

14 SET work_mem = '64kB';
15
16 -- Lab 3:
17 SET enable_hashjoin = OFF;
18 SET enable_nestloop = OFF;
19
20 EXPLAIN (ANALYZE, BUFFERS)
21 SELECT f.film_id, f.title, i.inventory_id
22 FROM film f
23 JOIN inventory i ON f.film_id = i.film_id
24 ORDER BY f.film_id;
25
26 RESET ALL;

```

Data Output Messages Notifications

Showing rows: 1 to 19 of 1

QUERY PLAN	
	text
1	Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=3.116..4.660 rows=4581.00 loops=1)
2	Merge Cond: (f.film_id = i.film_id)
3	Buffers: shared hit=87 read=26 dirtied=2, temp read=10 written=12
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.006..0.203 rows=1000.00 loops=1)
5	Index Searches: 1
6	Buffers: shared hit=84 read=1
7	-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=3.105..3.803 rows=4581.00 loops=1)
8	Storage: Memory Maximum Storage: 17kB
9	Buffers: shared hit=3 read=25 dirtied=2, temp read=10 written=12
10	-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=3.100..3.404 rows=4581.00 loops=1)
11	Sort Key: i.film_id
12	Sort Method: external merge Disk: 80kB
13	Buffers: shared hit=3 read=25 dirtied=2, temp read=10 written=12
14	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=1.062..1.819 rows=4581.00 loops=1)
15	Buffers: shared read=25 dirtied=2
16	Planning:
17	Buffers: shared hit=89 read=5
18	Planning Time: 14.108 ms
19	Execution Time: 11.929 ms

- Compare: Reset the settings (RESET ALL;) and remove the ORDER BY. Does it switch back to Hash Join? Why?

Nó có chuyển lại thành Hash Join, khi RESET ALL; và bỏ ORDER BY, Postgres dùng lại các thuật toán mặc định.